

Dijkstra?

Dijkstra's algorithm is a fundamental graph traversal technique designed to solve the single-source shortest path problem in weighted graphs with non-negative edge weights. The algorithm operates by systematically exploring vertices in order of their current best-known distance from the source vertex, gradually expanding the search frontier until all reachable vertices have been processed or the target vertex is found.

In this specific implementation for the given problem, the algorithm begins at vertex 1 and aims to reach vertex n through the shortest possible path. The solution employs several key data structures to achieve optimal performance: an adjacency list for efficient graph representation, a min-heap (priority queue) to always process the closest unvisited vertex next, distance and parent arrays to track optimal paths, and a visited set to avoid redundant processing. The adjacency list is particularly crucial for handling the large constraints (up to 100,000 vertices and edges), as it provides $O(1)$ access to a vertex's neighbors while using memory proportional to the actual number of edges.

The algorithm progresses through multiple iterations, each time extracting the vertex with the smallest known distance from the priority queue. For each extracted vertex, it examines all adjacent edges, updating the distances of neighboring vertices if a shorter path is discovered through the current vertex. A critical optimization includes early termination when the target vertex n is reached, preventing unnecessary computations. The parent array plays a vital role in path reconstruction, storing the predecessor of each vertex in the optimal path, which allows backtracking from vertex n to vertex 1 once the algorithm completes.

For the given example with vertices 1 through 5 and six edges, the algorithm correctly identifies the optimal path $1 \rightarrow 4 \rightarrow 3 \rightarrow 5$ with total weight 5, despite the existence of alternative routes like $1 \rightarrow 2 \rightarrow 5$ (weight 7) or $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$ (weight 7). The solution efficiently handles potential challenges such as multiple edges between the same vertices and self-loops by always selecting the minimum weight during relaxation. If no path exists between vertex 1 and n , the distance to n remains at the initialized large value (INF), triggering the output of -1 as specified in the problem requirements.

Steps for Dijkstra:

Phase 1: Setup and Initialization

1. Graph Representation

- Represent the graph as connections between vertices with their weights
 - Each vertex knows its neighbors and the cost to reach them
2. **Tracking Structures**
- Create a distance tracker for each vertex (initially infinity for all except start)
 - Create a predecessor tracker to remember the path
 - Create a visited set to mark processed vertices
 - Create a priority queue that always gives the closest unvisited vertex

Phase 2: Algorithm Execution

3. **Start from Source**
- Set distance of starting vertex (vertex 1) to 0
 - Add starting vertex to the priority queue with distance 0
4. **Main Processing Loop**
- **Select:** Take the vertex with smallest known distance from the queue
 - **Mark:** Mark this vertex as visited (so we don't process it again)
 - **Check Destination:** If this vertex is the target (vertex n), we can stop early
 - **Explore Neighbors:** For each connected vertex from current vertex:
 - Calculate the total distance to reach that neighbor through current vertex
 - If this new distance is shorter than previously known:
 - Update the neighbor's distance
 - Remember current vertex as the predecessor for this neighbor
 - Add the neighbor to the priority queue with its new distance

Phase 3: Result Determination

5. **Path Existence Check**
- If the target vertex's distance is still infinity → no path exists
 - Otherwise, a shortest path exists
6. **Path Reconstruction** (if path exists)
- Start from the target vertex (vertex n)
 - Repeatedly follow the predecessor pointers backward
 - Collect all vertices along this backward journey

- Reverse the sequence to get the path from start to end

Input:

5 6

1 2 2

2 5 5

2 3 4

1 4 1

4 3 3

3 5 1

Execution:

Step 1: Read Input

- Read $n = 5$, $m = 6$
- Create an empty graph with 6 positions (for vertices 1 to 5)

Step 2: Build Graph

- For edge 1-2 weight 2: connect vertex 1 to 2 with weight 2, and vertex 2 to 1 with weight 2
- For edge 2-5 weight 5: connect vertex 2 to 5 with weight 5, and vertex 5 to 2 with weight 5
- For edge 2-3 weight 4: connect vertex 2 to 3 with weight 4, and vertex 3 to 2 with weight 4
- For edge 1-4 weight 1: connect vertex 1 to 4 with weight 1, and vertex 4 to 1 with weight 1
- For edge 4-3 weight 3: connect vertex 4 to 3 with weight 3, and vertex 3 to 4 with weight 3

- For edge 3-5 weight 1: connect vertex 3 to 5 with weight 1, and vertex 5 to 3 with weight 1

Step 3: Initialize Data

- Distance array: vertex 1=0, vertices 2-5=infinity
- Parent array: all vertices = -1 (no parent)
- Visited array: all vertices = false (not visited)
- Priority queue: add vertex 1 with distance 0

Step 4: Process Vertex 1

- Remove vertex 1 from queue (distance 0)
- Mark vertex 1 as visited
- Check neighbor vertex 2: new distance $0+2=2$, update distance to 2, set parent to 1, add to queue
- Check neighbor vertex 4: new distance $0+1=1$, update distance to 1, set parent to 1, add to queue
- Queue now has: vertex 4 (distance 1), vertex 2 (distance 2)

Step 5: Process Vertex 4

- Remove vertex 4 from queue (distance 1)
- Mark vertex 4 as visited
- Check neighbor vertex 1: already visited, skip
- Check neighbor vertex 3: new distance $1+3=4$, update distance to 4, set parent to 4, add to queue
- Queue now has: vertex 2 (distance 2), vertex 3 (distance 4)

Step 6: Process Vertex 2

- Remove vertex 2 from queue (distance 2)
- Mark vertex 2 as visited
- Check neighbor vertex 1: already visited, skip
- Check neighbor vertex 5: new distance $2+5=7$, update distance to 7, set parent to 2, add to queue
- Check neighbor vertex 3: new distance $2+4=6$, but current distance is 4, so no update
- Queue now has: vertex 3 (distance 4), vertex 5 (distance 7)

Step 7: Process Vertex 3

- Remove vertex 3 from queue (distance 4)
- Mark vertex 3 as visited
- Check neighbor vertex 2: already visited, skip
- Check neighbor vertex 4: already visited, skip
- Check neighbor vertex 5: new distance $4 + 1 = 5$, better than current 7, update distance to 5, set parent to 3, add to queue
- Queue now has: vertex 5 (distance 5), vertex 5 (distance 7)

Step 8: Process Vertex 5

- Remove vertex 5 from queue (distance 5)
- Mark vertex 5 as visited
- Vertex 5 is the target, so stop the algorithm

Step 9: Check Result

- Distance to vertex 5 is 5 (not infinity), so path exists

Step 10: Reconstruct Path

- Start from vertex 5
- Parent of 5 is 3
- Parent of 3 is 4
- Parent of 4 is 1
- Path backwards: 5, 3, 4, 1
- Reverse path: 1, 4, 3, 5

Step 11: Output Result

- Print: 1 4 3 5

Output:

1 4 3 5

Pseudocode:

READ n, m

CREATE $\text{graph}[n+1]$

FOR each edge:

 READ a, b, w

 ADD (b, w) to $\text{graph}[a]$

 ADD (a, w) to $\text{graph}[b]$

$\text{dist} = [\text{INF}] * (n+1)$

$\text{parent} = [-1] * (n+1)$

$\text{visited} = [\text{false}] * (n+1)$

$\text{pq} = \text{min-heap}$

$\text{dist}[1] = 0$

PUSH $(0, 1)$ to pq

WHILE pq not empty:

$(d, u) = \text{POP from pq}$

 IF $\text{visited}[u]$: CONTINUE

$\text{visited}[u] = \text{true}$

IF $u == n$: BREAK

FOR each (v, w) in $\text{graph}[u]$:

IF not visited $[v]$ AND $d + w < \text{dist}[v]$:

dist $[v] = d + w$

parent $[v] = u$

PUSH (dist $[v]$, v) to pq

IF dist $[n] == \text{INF}$:

PRINT -1

ELSE:

path = []

$v = n$

WHILE $v \neq -1$:

ADD v to path

$v = \text{parent}[v]$

REVERSE path

PRINT path